

Day 4 - Your First Lightning Web Component (LWC)

Part 1 - Think Before Coding

1. Why do users need a graphical interface?

Users need a Graphical User Interface (GUI) because it makes Salesforce easy to use without requiring programming knowledge.

A GUI allows users to:

- Click buttons
- Fill out forms
- Search for records
- View information in tables
- Navigate between different pages

Instead of writing code or database queries, users can interact with the application visually.

2. Why can't users directly execute SOQL queries?

Normal users should not execute SOQL queries directly because:

- They may not know SOQL syntax.
- Incorrect queries may cause errors.
- Complex queries may affect performance.
- Writing queries is not user-friendly.
- Data access must follow Salesforce security rules.

Instead, the application retrieves the required records and displays them through a simple user interface.

3. Why is JavaScript required in LWC?

JavaScript is used in Lightning Web Components to control the logic and behavior of the component.

JavaScript is used to:

- Handle button clicks
- Respond to user actions
- Store and process data
- Validate user input
- Call Apex methods

- Update the UI dynamically
- Handle events

Easy way to remember:

HTML = Structure / UI

JavaScript = Logic / Behavior

4. What responsibilities belong to the UI?

The UI is mainly responsible for presenting information and interacting with users.

UI responsibilities include:

- Displaying records and information
- Taking input from users
- Showing buttons, forms, and tables
- Performing basic client-side validation
- Sending requests to Apex when required
- Displaying success and error messages
- Providing an easy-to-use experience

5. Which responsibilities should remain in Apex?

Apex should handle server-side processing, database operations, and complex business logic.

Apex responsibilities include:

- Executing SOQL queries
- Performing DML operations such as Insert, Update, and Delete
- Implementing complex business rules
- Processing large amounts of data
- Managing transactions
- Applying appropriate security and data-access logic
- Providing data to LWC components

Part 2 - Understanding LWC

A Lightning Web Component normally contains three important files.

1. studentHome.html - HTML

The HTML file defines the user interface and structure of the component.

It handles:

- Page layout
- Headings and text
- Buttons
- Images
- Input fields
- Forms
- Displaying records and other data

Example:

A Student Home Page can display a welcome message, student details, and buttons such as View Applications and View Profile.

2. studentHome.js - JavaScript

The JavaScript file controls the logic and behavior of the component.

It handles:

- Variables
- Component logic
- Events
- Button-click actions
- User input
- Data processing
- Communication with Apex

Example:

When the student clicks the View Applications button, JavaScript handles the action and performs the required logic.

3. studentHome.js-meta.xml - Meta XML

The Meta XML file defines where and how the Lightning Web Component can be used in Salesforce.

It is responsible for:

- Making the component available in Lightning App Builder
- Exposing the component to Lightning pages
- Defining supported page types
- Setting targets such as App Page, Home Page, and Record Page

File	Responsibility
studentHome.html	What the user sees
studentHome.js	What the component does
studentHome.js-meta.xml	Where the component can be used

Simple Summary

HTML → UI

JavaScript → Logic

Meta XML → Configuration and availability

Part 3 - Hands-on Activity 1

Objective

Create the first Lightning Web Component (LWC) for the Placement Management System and deploy it on a Lightning Page.

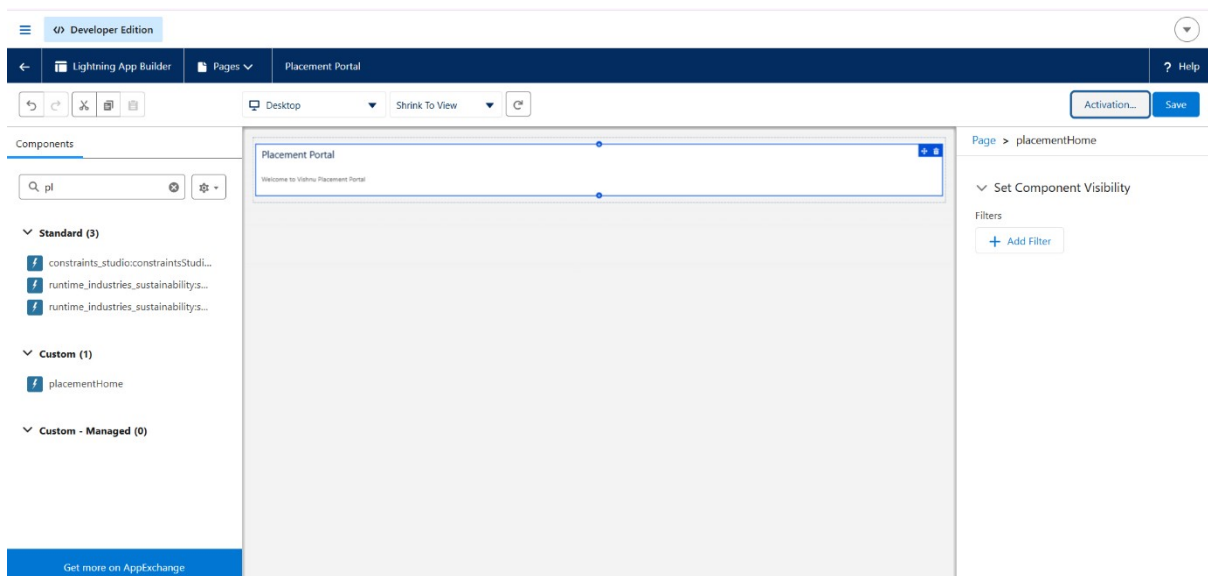
Component Name

placementHome

Output

Placement Portal

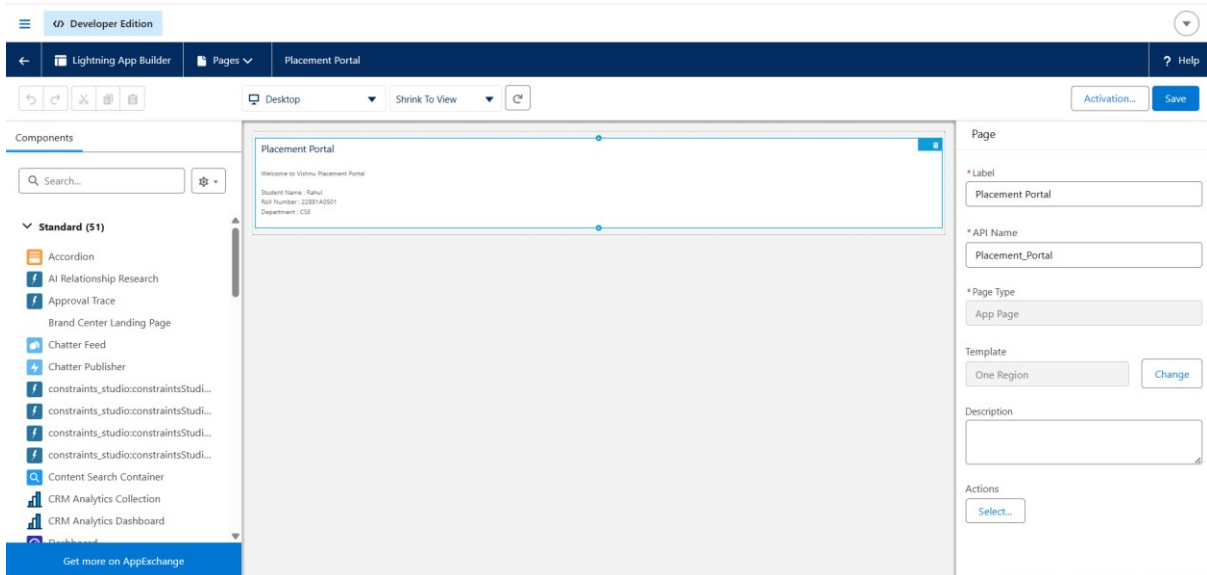
Welcome to Vishnu Placement Portal.



Part 4 - Hands-on Activity 2

Objective

To create Student Name, Roll Number, and Department variables in an LWC, display their values on the screen, and observe how the UI changes when the variable values are updated.



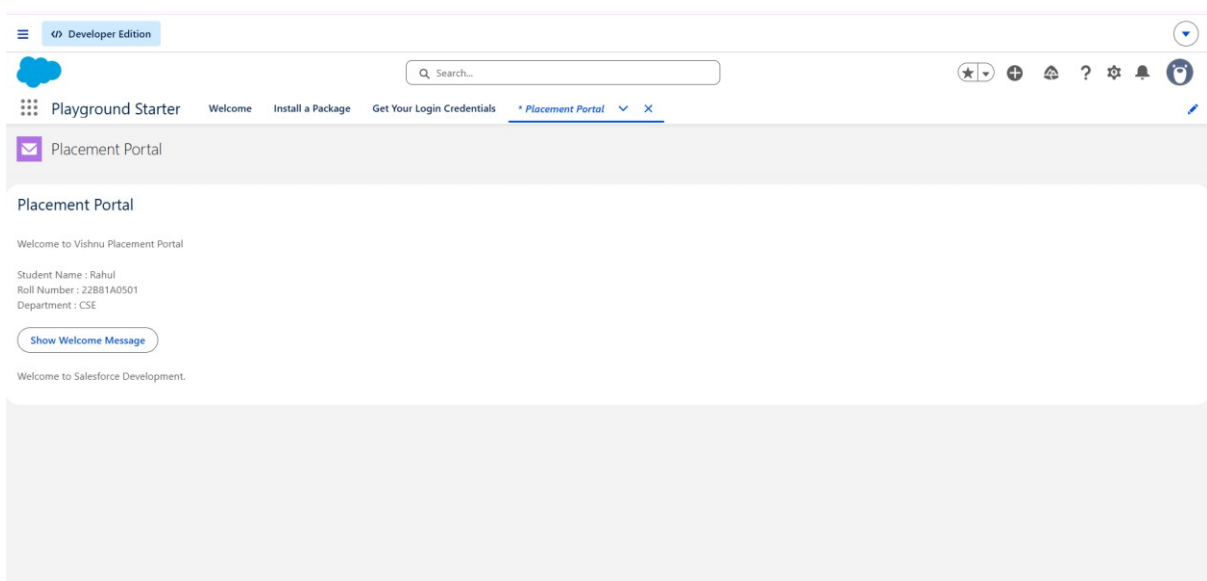
Part 5 - Hands-on Activity 3

Objective

To create a button in an LWC, handle a click event using JavaScript, and display a welcome message dynamically.

Initially, only the Show Welcome Message button appears.

After clicking the button, it displays: Welcome to Salesforce Development.



Part 6 - Hands-on Activity 4

Objective

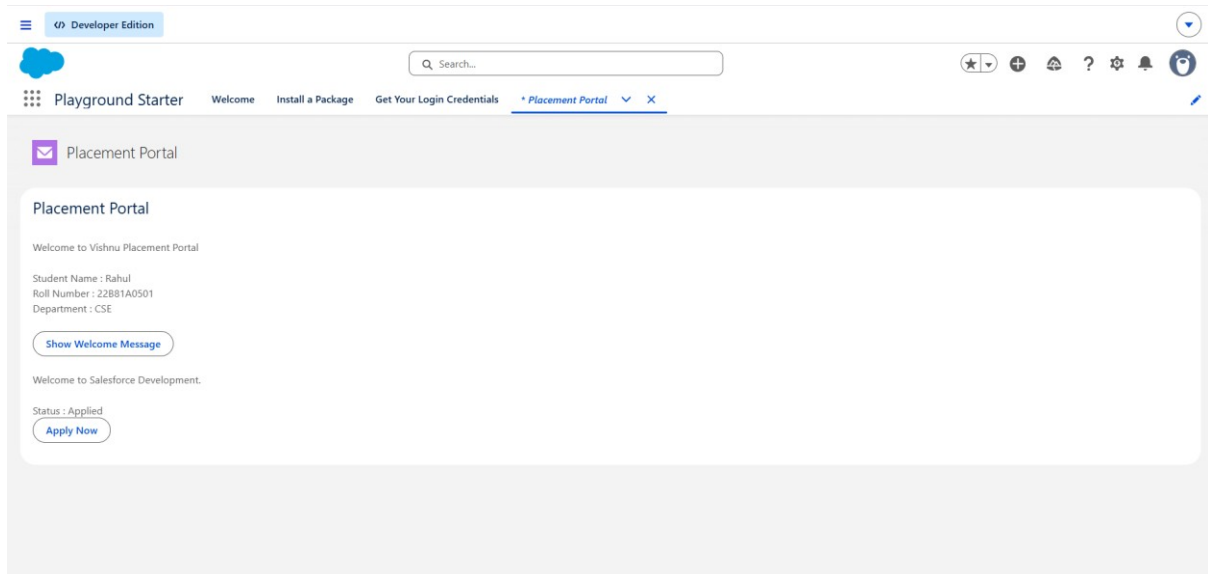
To understand how JavaScript can dynamically change data in an LWC without using Apex or a database.

Before clicking the button:

Status : **Not Applied**

After clicking Apply

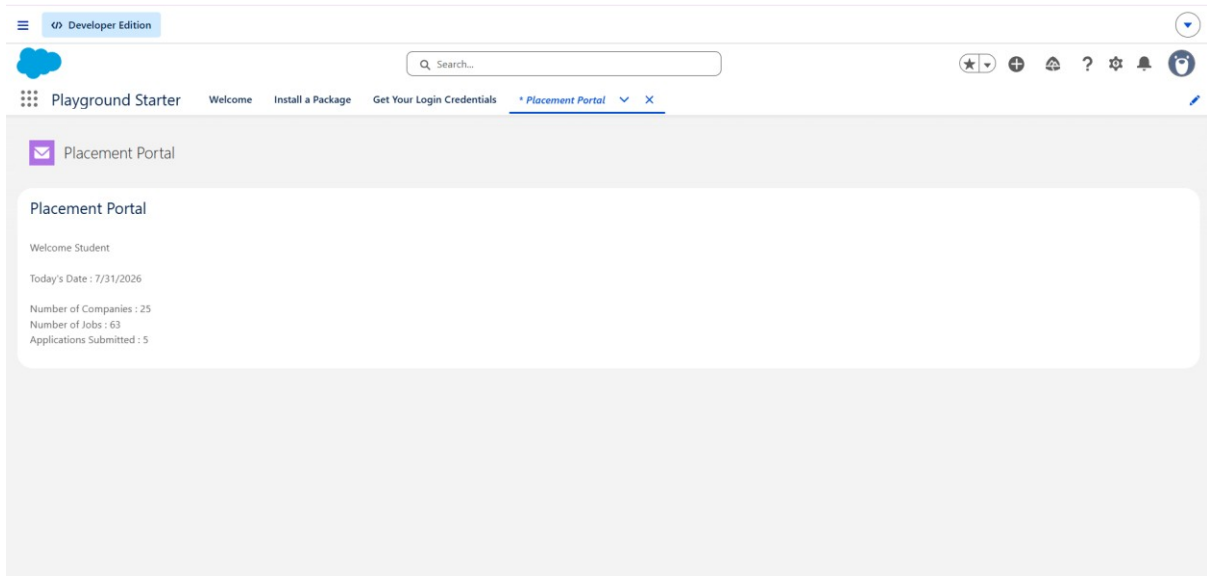
Status : **Applied**



Part 7 - Mini Project Enhancement

Objective

To create the first screen of the Placement Management System using LWC and display placement information using hard-coded JavaScript values.



Part 8 - Understanding Data Binding

Objective

To understand data binding in LWC by displaying a JavaScript variable in HTML and observing how the UI changes when the variable value is changed.

